



GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING & TECHNOLOGY

(Autonomous)

Department of Electronics and Communication Engineering

Adaptive Multi-Source End-to-End Communication System

FINAL PRESENTATION

02 May 2026

**Under the Guidance
of**

Dr. Hima Bindu V, ECE Dept, GRIET

Batch no. 6

Ajasra 24241A0401

Sameena 24241A0455

Hari 24241A0440

CONTENTS

Introduction

Problem Statement

Objectives & Outcomes

Proposed Solution

Components & Software Used

Block Diagram / System Architecture

Steps 1–2: Signal Generation & Pre-Processing

Step 3: Signal Detection

Step 4: Demodulation

Steps 5–6: Message Extraction & ADC

Step 7: Encoding (Error Control Coding)

Step 8: Digital Modulation

Step 9: Channel (AWGN)

Steps 10–11: Receiver Demodulation & Decoding

Step 12: DAC & Output + Adaptive Feedback

Results & Discussion

Real-World Applications & Limitations

Conclusion, Future Scope & References



INTRODUCTION

Introduction / Background

Modern communication systems must handle multiple signal types: Clean, AM, FM, Noise, AM+Noise, FM+Noise. Each type requires a completely different processing approach.

Why This Domain Matters

Communication is the backbone of IoT, 5G, satellite, and wireless systems. Adaptive systems are crucial for efficient spectrum usage and low error rates.

Core Concept

Traditional systems use fixed processing pipelines. This project implements an intelligent, self-adapting pipeline that:

- Detects signal type automatically
- Selects the optimal processing technique in real time
- Recovers the original message with minimum error



PROBLEM STATEMENT

Limitations of Existing Systems

Fixed system design — cannot adapt to changing signal types
Inability to automatically detect the incoming signal type
Performance degrades severely in noisy environments
Increased bit errors and signal distortion under varying conditions

Core Problem

Lack of adaptability leads to inefficient signal recovery. Systems fail to maintain reliable communication under varying conditions.

Real-World Relevance

In 4G/5G, WiFi, and IoT systems — signals are never clean and static. A fixed-processing system cannot handle dynamic environments. This creates a critical need for adaptive, intelligent communication systems.

OBJECTIVES AND OUTCOMES

Project Objectives

Design a complete end-to-end adaptive communication system

Automatically detect signal type (Clean / AM / FM / Noise / AM+Noise / FM+Noise)

Implement adaptive filtering, demodulation, ADC, and modulation

Use BER-based feedback to optimise transmission parameters

Demonstrate the system in MATLAB using Audio, Equation, and ESP32 sources

Outcomes

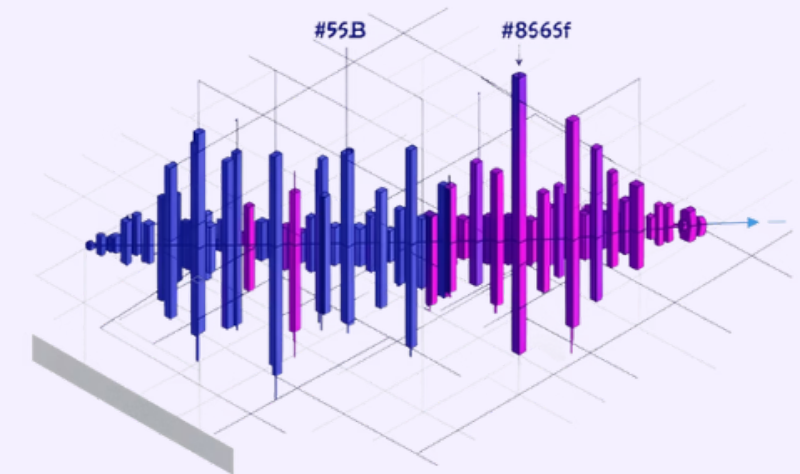
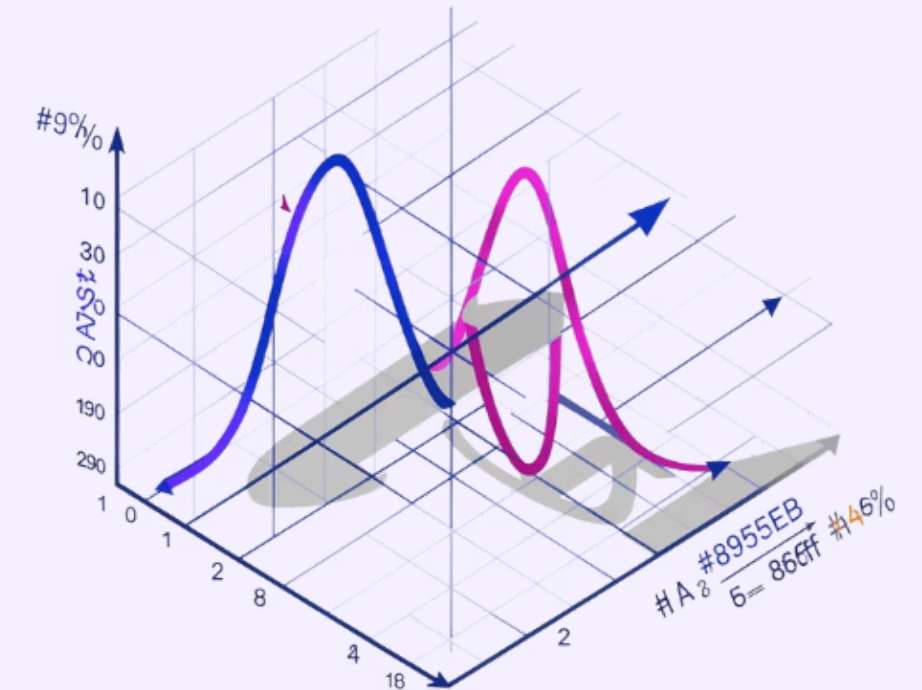
Improved signal quality after end-to-end processing

Reduced Bit Error Rate (BER) through adaptive techniques

Robust performance under noisy channel conditions

Feedback-based system that self-corrects based on BER

A single system that handles 6 signal types across 3 source types via a 12-step pipeline



PROPOSED SOLUTION

System Name: Adaptive Multi-Source End-to-End Communication System

Key Capability : Adaptive Processing Based on Signal Conditions

The system automatically selects processing techniques based on detected signal type, with BER-based feedback for continuous optimization.

Supported Input Sources

Audio Signals — real audio file (song / voice)

Mathematical Signals — equation-generated ideal signals

ESP32 Sensor Signals — real-time hardware signals

End-to-End Pipeline (12 Steps)

Input Generation → Pre-Processing → Signal Detection → Demodulation → Message Extraction → ADC → Encoding → Digital Modulation → Channel → Receiver Demodulation → Decoding → DAC & Output

Advantages Over Existing Methods

Self-adapting — no manual reconfiguration needed

Handles all 6 signal types in a single unified pipeline

BER feedback loop ensures continuous quality improvement

COMPONENTS & SOFTWARE USED

Hardware Components

- ESP32 Development Board: Microcontroller for real-time signal acquisition
- Potentiometer: Generates analog input (clean signal)
- Gas Sensor (MQ Series): Used to simulate noise signal
- MPU6050 (Accelerometer + Gyroscope): Provides dynamic signal for FM modulation
- Breadboard & Jumper Wires: Circuit connections
- USB Cable: Power + Serial communication

Esp32 Roles

Potentiometer → Clean Signal $m(t)$

Gas Sensor → Noise Signal (random fluctuations)

MPU6050 → FM Control (motion/tilt = frequency change via $\int a_x dt$)

Software Tools

MATLAB — Core platform for signal processing & system implementation

Arduino IDE — Programming ESP32 and sensor interfacing

Serial Communication (UART) — Data transfer between ESP32 and MATLAB

ARCHITECTURE

End-to-End 12-Step Adaptive Communication Pipeline

Step 1 — Input Signal Generation: Audio / Equation / ESP32 sensors

Step 2 — Pre-Processing: Filtering (LPF / BPF / HPF / Notch) + Signal Normalization

Step 3 — Signal Detection: Feature Analysis

Step 4 — Demodulation : AM: Envelope Detection | FM: Phase-Locked Loop (PLL)

Step 5 — Message Extraction

Step 6 — ADC :LPF + Carrier Removal → Flash / SAR / Sigma-Delta ADC (adaptive)

Step 7 — Encoding → Step 8 — Digital Modulation: Parity / Hamming → BPSK / QPSK / 16-QAM (adaptive)

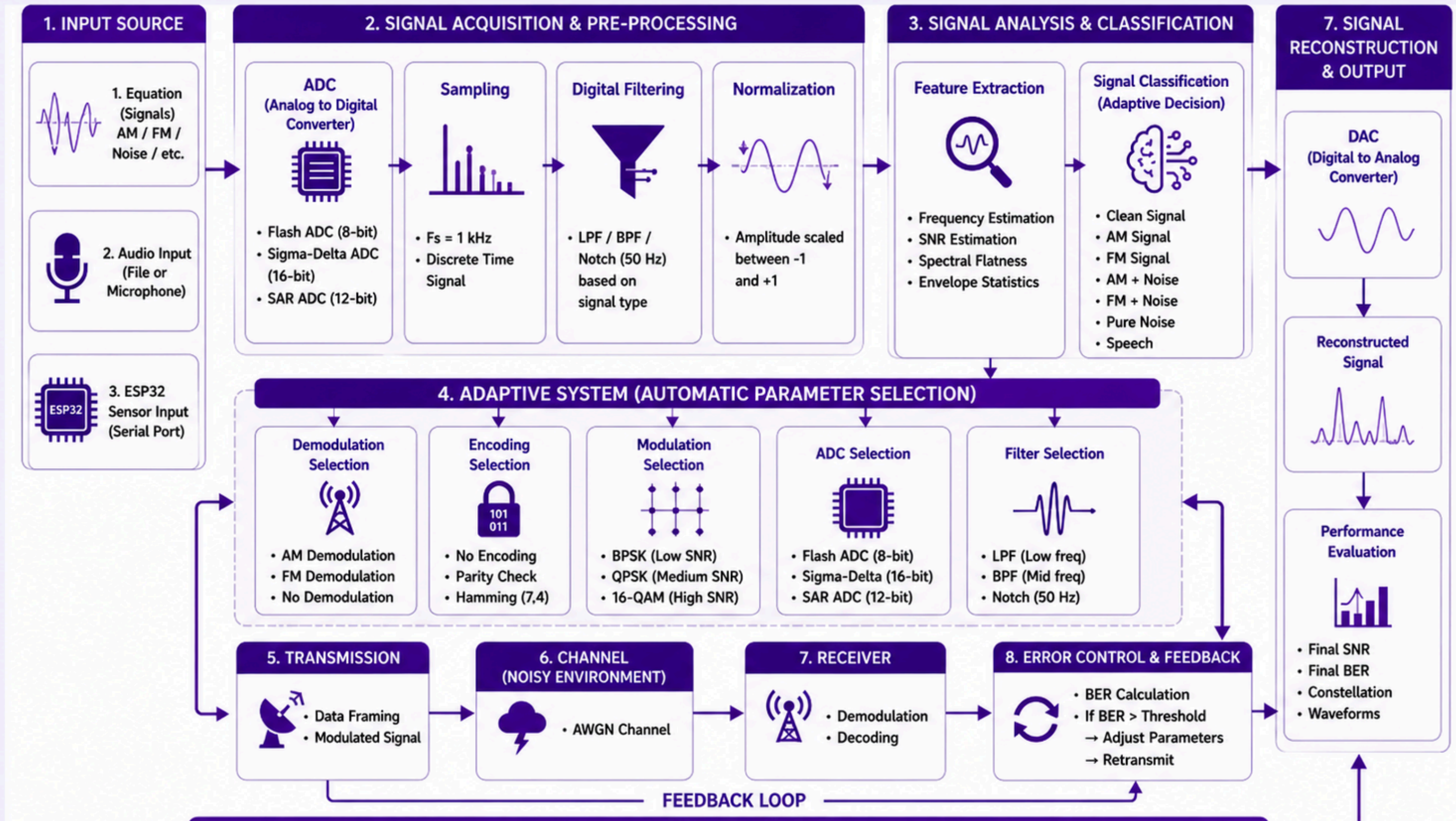
Step 9 — Channel (AWGN): Noise added; SNR determines signal quality

Step 10 — Receiver Demodulation → Step 11 — Decoding ,Recover bits from signal → Error detection & correction

Step 12 — DAC & Output: Bits → analog waveform → reconstructed signal

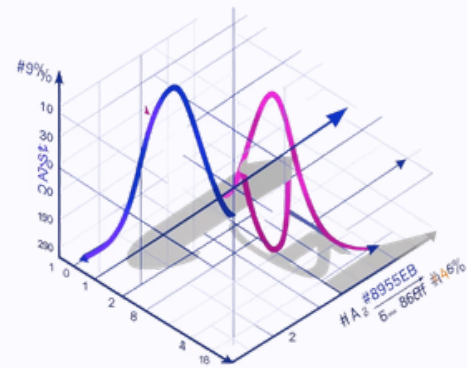
Adaptive Feedback Loop: BER Calculation → Adjust Modulation / Encoding / Retransmit

System Architecture: 12-Step Pipeline



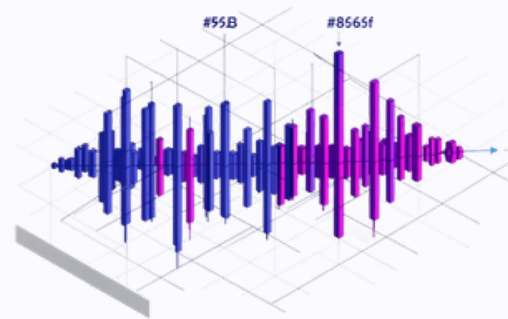
STEP 1 : Input Signal Generation

One original source is transformed into **6 signal types**: clean, AM, FM, noisy, AM+noise, FM+noise.



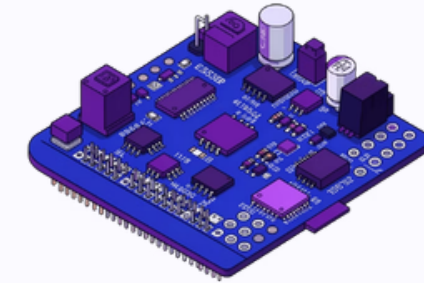
Equation Signals

MATLAB-generated: pure sine, AM envelope, FM phase, and noise-corrupted variants using `randn()`.



Audio Signals

Original audio file → AM:
 $(1+\mu x) \cdot \cos(2\pi fct)$, FM: $\cos(2\pi fct + kf \cdot x)$,
plus noise variants.



ESP32 Sensors

Potentiometer → message $m(t)$;
MPU6050 → FM control via `vacumsum(ax)`;
gas sensor → noise.

Step 2 & 3: Pre-Processing & Signal Detection

Pre-Processing

1) Filtering

Low Pass Filter (LPF) — Used for: clean signal ,extraction

Band Pass Filter (BPF) —Used for: AM / FM signals

High Pass Filter (HPF) — Keeps high-frequency components

Notch Filter — Removes a specific frequency (e.g., 50 Hz noise)

2) Normalisation

Adjusts amplitude to standard range: -1 to $+1$.

Prevents distortion, makes processing stable, improves ADC and modulation.

Feature-Based Detection

Four features classify each signal:

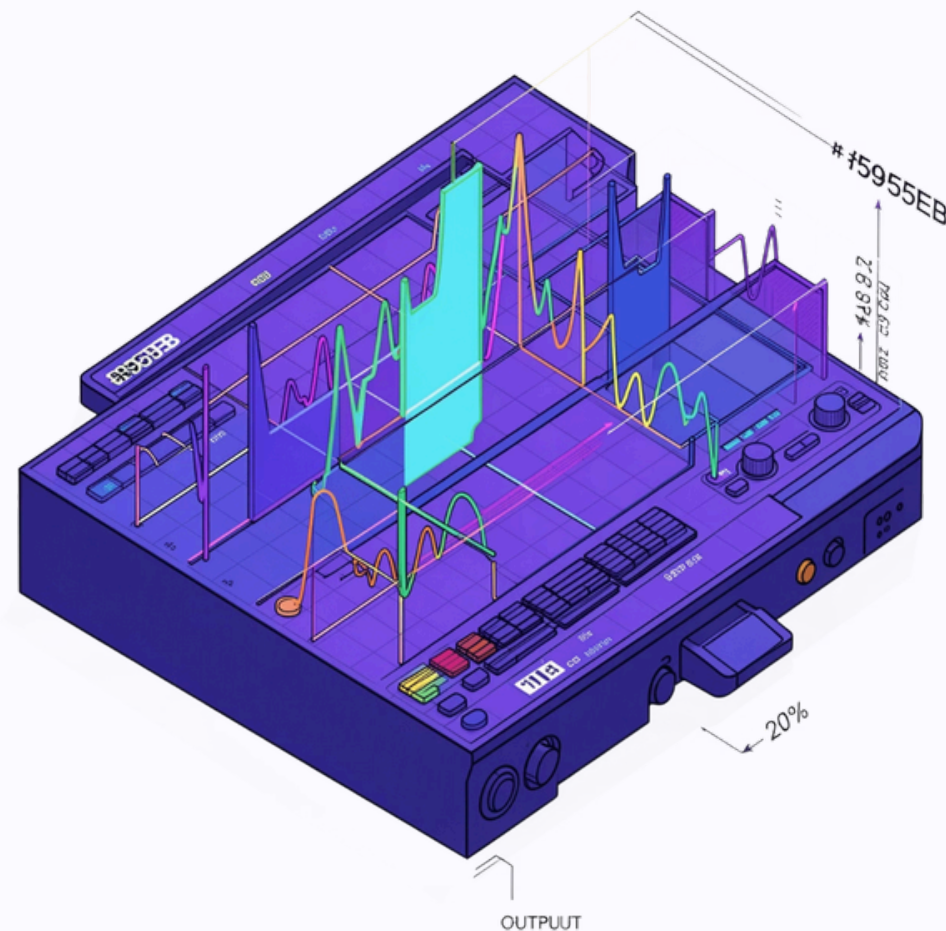
Envelope Variance: `analytic = ifft(fft(x).*H);`
`env = abs(analytic);`
`env_var = std(env - mean(env));`

Spectral Flatness: Measures how evenly energy is distributed across frequencies.

Noise Level: `noise_level = std(x - env);`
(Low : Clean ,High: Noisy signal)

Frequency Variation: `phase = unwrap(angle(analytic));`
`diff_std = std(diff(phase));`

STEP 4 : DEMODULATION



AM Demodulation

Coherent detection: multiply by carrier $\rightarrow$ LPF extracts message $m(t)$.

FM Demodulation

Phase Locked Loop (PLL) tracks frequency variations to recover the original signal.

Message Extraction LPF

A **second LPF** removes carrier remnants ($2f_c$) created during demodulation, isolating the clean baseband message.

Memory trick: Pre-filter $\rightarrow$ clean input. Post-filter $\rightarrow$ extract message.

STEPS 5 & 6: MESSAGE EXTRACTION & ADC

Step 5: Message Extraction

After demodulation, signal still contains unwanted high-freq components. Apply LPF again to extract only the baseband message.

Low-Pass Filtering — removes residual high-frequency noise
Carrier Removal — removes carrier component for cleaner output

Note:

Adaptive Selection:

High SNR → Flash
Medium SNR → SAR
Low SNR → Sigma-Delta

Step 6: Analog to Digital Conversion (ADC)

Converts continuous signal → binary data (0s & 1s).

Three ADC types implemented adaptively:

Flash ADC: Ultra-fast, uses multiple parallel comparators.
`levels = linspace(-1,1,2^n); [~,idx] = min(abs(x - levels));`
Used when: High SNR (clean signal)

SAR ADC : Binary search algorithm.
Medium speed | Good accuracy | Balanced complexity
Used when: Medium SNR

Sigma-Delta ADC : Oversampling + feedback (noise shaping).
Very high accuracy | Low noise | Slower speed
Used when: Low SNR (noisy signal)

STEP 7: ENCODING (ERROR CONTROL CODING)

Adds extra (redundant) bits to detect and correct errors. Noise causes bit flips (0 ↔ 1) during transmission.

Example: Sent: 101101 | Received: 100101 ✗

Adaptive Selection: High SNR → Parity (simple, fast) | Low SNR → Hamming (robust)

Parity Encoding

Adds a single bit for error detection.

Example: Data: 1011 (3 ones = odd) → Add 1 → 10111 (even parity)

At Receiver: Count 1s — if not even → error detected.

Hamming Code (7,4)

Detects AND corrects single-bit errors.

Data: D1 D2 D3 D4 = 1 0 1 1

Insert parity at positions 1, 2, 4:

Position: 1 2 3 4 5 6 7

P1 P2 D1 P3 D2 D3 D4

Calculate P1, P2, P3 using XOR checks → Final codeword: 0 1 1 0 0 1 1

At Receiver: Compute syndrome S1, S2, S3

Error position = binary value of S3 S2 S1 (e.g., 011 = position 3)

Flip that bit → data recovered correctly ✓

STEP 8: DIGITAL MODULATION

Adaptive Modulation Selection: Raw bits → Symbols → Waveform → Transmit. Bits cannot be sent over air directly.

BPSK

1 bit → 1 symbol | 0 → -1 (180°) | 1 → +1 (0°)

symbols = 2*bits - 1; Signal: $s(t) = \pm \cos(2\pi fct)$

High noise resistance | Low speed

Used when: Low SNR

QPSK

2 bits → 1 symbol (4 phases: 45°, 135°, -45°, -135°)

I = bits(1:2:end); Q = bits(2:2:end); symbols = I + 1j*Q;

Medium speed | Medium noise resistance

Used when: Moderate SNR

16-QAM

4 bits → 1 symbol. Uses both amplitude AND phase variation.

I = levels(bits(1:2:end));

Q = levels(bits(3:4:end));

symbols = I + 1j*Q;

High speed | Low noise resistance

Used when: High SNR

Adaptive Selection Logic

Low SNR → BPSK

Moderate SNR → QPSK

High SNR → 16-QAM

System continuously monitors channel condition and switches accordingly.

STEP 9: CHANNEL

AWGN Channel : Medium through which signal travels — Wireless (air), Cable, Optical fiber.

AWGN: Additive White Gaussian Noise : Additive — Received = Signal + Noise
White — Affects all frequencies equally
Gaussian — Follows normal (bell curve) distribution

MATLAB:

```
noise = randn(size(signal));  
rx = signal + noise_scaled;
```

Signal-to-Noise Ratio (SNR) : $SNR = \text{Signal Power} / \text{Noise Power}$

High SNR Signal $\gg$ Noise Clear transmission, low errors → use 16-QAM

Medium SNR Signal $\approx$ Noise Some distortion, moderate errors → use QPSK

Low SNR Noise $>$ Signal High distortion, high errors → use BPSK

Effect on System: Noise causes: bit errors, signal distortion, reduced accuracy.

Adaptive response: High noise → BPSK | Low noise → 16-QAM

Step 10 & 11: RECEIVER DEMODULATION & DECODING

Receiver Demodulation

Noisy Signal → Demodulation → Recovered Bits. Reverses the modulation process.

BPSK Demodulation: $0 \rightarrow -1 \mid 1 \rightarrow +1$
After channel: +1 may become +0.7, -1 may become -0.3
 $\text{rx_bits} = \text{real}(\text{rx_syms}) > 0$
Value $> 0 \rightarrow \text{bit} = 1$, Value $< 0 \rightarrow \text{bit} = 0$

QPSK Demodulation: $I = \text{real}(\text{rx_syms});$
 $Q = \text{imag}(\text{rx_syms});$
 $\text{bit1} = I > 0, \text{bit2} = Q > 0$

Quadrant of symbol determines output bits.

16-QAM Demodulation: Select closest constellation point to received symbol (minimum distance detection).
 $[\sim, \text{idx}] = \min(\text{abs}(\text{rx_symbol} - \text{constellation_points}));$

Example: Received $(2.6 + j0.8) \rightarrow$ Closest point $(3 + j1) \rightarrow$ Output bits: 1011
Error occurs if noise shifts signal into wrong constellation region.

Decoding

Received Bits → Decoding → Corrected Bits.

Parity Check — detects errors, cannot correct

Hamming Decode — corrects single-bit errors without retransmission

Compute syndrome → find error position → flip bit → data recovered

Adaptive:

High SNR → Parity
Low SNR → Hamming

STEP 12: DAC & OUTPUT + ADAPTIVE FEEDBACK

DAC & Output

Digital Bits → DAC → Analog Signal. Converts digital data back into a continuous waveform.

Staircase Waveform — discrete steps that approximate the analog signal

Reconstruction Filter — smooths the staircase, removes high-freq artifacts

Final Output — closely resembles the original input signal

Adaptive Feedback System

Receiver sends performance info back to Transmitter, enabling the system to adapt based on quality.

BER Calculation: $BER = \text{Number of wrong bits} / \text{Total bits}$

Example: Sent: 101101 | Received: 101001 → Errors = 1 → $BER = 1/6$

Adaptive Feedback Process

Receiver checks BER → Low BER = good | High BER = poor

Decision: Low BER → keep settings | High BER → adjust system

Adjustments:

Modulation switched (BPSK/QPSK/16-QAM) |
Encoding switched (Parity/Hamming) |
Retransmission triggered if error rate too high

Without Feedback	With Feedback
Fixed system	Adaptive system
Cannot handle noise	Reduces errors
High error rate	Improves performance

RESULT:

ADAPTIVE COMMUNICATION SYSTEM v5.2 [UNIFIED]

SELECT INPUT SOURCE

- 1 - My own equation (AM/FM/noise/etc.)
- 2 - Audio Input (file or microphone)

Enter 1 or 2 : 1

SELECT SIGNAL TYPE

- 1 Clean Sine
- 2 AM Signal
- 3 FM Signal
- 4 AM + Noise
- 5 FM + Noise
- 6 Pure Noise
- 7 Composite (multi-tone)
- 8 Custom Equation

Enter 1-8 : 3

[OK] Signal generated: FM $f_c=100\text{Hz}$ $f_m=10\text{Hz}$ $b=0.8$ (1000 samp, $F_s=1000\text{Hz}$)

INPUT SIGNAL ANALYSIS REPORT

SIGNAL IDENTIFICATION

Input Label : FM $f_c=100\text{Hz}$ $f_m=10\text{Hz}$ $b=0.8$
Detected Type : FM Signal
Est. Carrier Freq : 100.00 Hz

TIME DOMAIN PARAMETERS

Samples : 1000
Sample Rate (F_s) : 1000 Hz
Duration : 1.0000 s
Peak Amplitude : 1.0000
RMS Amplitude : 0.7074
Mean Value : 0.000000
Std Deviation : 0.7077
Zero Crossings : 199 (est. 99.5 Hz)
Positive Peaks : 100
Negative Peaks : 100

FREQUENCY DOMAIN PARAMETERS

Est. Carrier Freq : 100.00 Hz
Spectral Flatness : 0.0000 (0=tonal, 1=noise)
Envelope Variance : 0.0000
Frequency Spread : 0.0113

SIGNAL QUALITY METRICS

Estimated SNR : 172.53 dB
Noise Level (flat) : 0.0000
Smoothing MSE : 0.505208

ADAPTIVE PIPELINE DECISIONS (pre-SNR)

Filter Applied : BPF 75-125Hz
Demod Technique : FM PLL Demodulator
ADC Method : Flash ADC (8-bit)

CHANNEL SNR SETTING

< 8 dB → BPSK

8-15 dB → QPSK

> 15 dB → 16-QAM

Enter SNR in dB [12] : 50

DIGITAL PIPELINE DECISIONS (post-SNR)

SNR Entered : 12.0 dB

Encoding Method : Even Parity (8,9)

Digital Modulation : QPSK (SNR 8-15 dB)

— STEP 2 : Adaptive Filter —

[AUTO] Filter : BPF 75-125Hz

— STEP 3 : Adaptive Demodulation —

[AUTO] Demod : FM PLL (auto: Kp=0.1500 Ki=0.002000 BW=5.0Hz)

— STEP 5 : Adaptive ADC —

[AUTO] ADC : Flash ADC (8-bit)

— STEP 6 : Adaptive Encoding —

[AUTO] Encode : Even Parity (8,9) (8000→9000 bits)

— STEP 7-9 : DigMod → Channel → RX Demod —

[AUTO] DigMod : QPSK (SNR 8-15 dB)

— STEP 10 : Adaptive Decoding —

[AUTO] Decode : Parity Decode | Errors corrected: 0

— FEEDBACK LOOP —

Initial BER : 0.00000 (threshold = 0.0100)

[FB] ✓ BER acceptable after 0 retx

— STEP 11 : DAC Reconstruction —

[AUTO] DAC : Linear Interp DAC (8-bit, 1000 samples)

— STEP 12 : Output —

[OK] Recovery SNR : 32.77 dB

[OK] Final BER : 0.00000

[OK] Retransmits : 0

[OK] Final Mod : QPSK (SNR 8-15 dB)

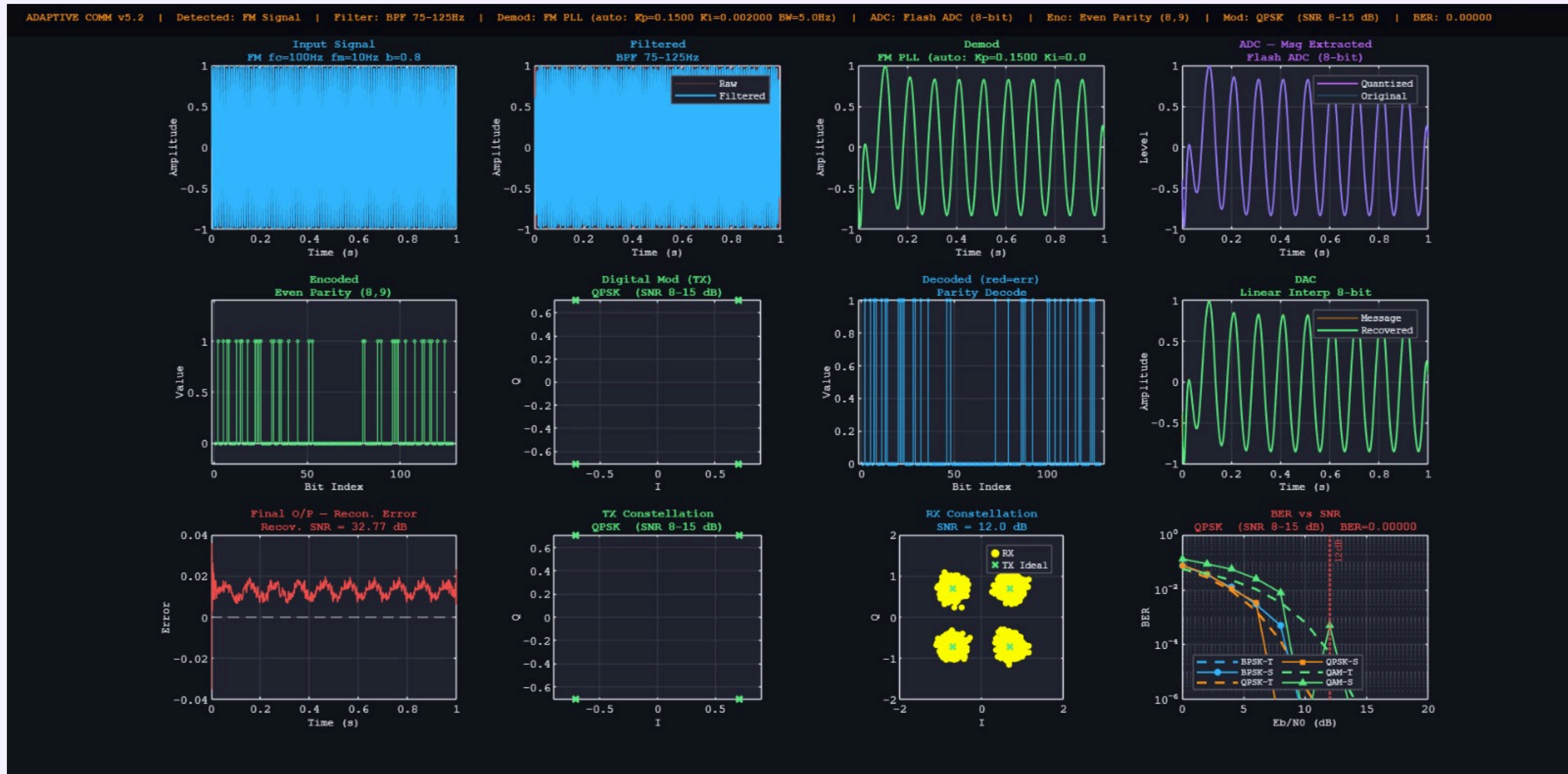
FINAL PIPELINE SUMMARY

STEP	STAGE	METHOD USED
1 Input	Signal Source	FM $f_c=100\text{Hz}$ $f_m=10\text{H}$
2 Filter	Adaptive Filter	BPF 75-125Hz
3 Demod	Demodulation	FM PLL (auto: $K_p=0$)
4 Msg	Msg Extraction	LPF @250Hz
5 ADC	Analog→Digital	Flash ADC (8-bit)
6 Encode	Channel Encoding	Even Parity (8,9)
7 Mod(TX)	Digital Mod	QPSK (SNR 8-15 dB)
8 Channel	AWGN Channel	SNR=12.0dB
9 Demod RX	RX Demodulation	QPSK (SNR 8-15 dB)
10 Decode	Channel Decoding	Parity Decode
11 DAC	Digital→Analog	Linear Interp 8b

PERFORMANCE METRICS

Detected Signal Type : FM Signal
 Operating SNR : 12.0 dB
 Measured TX SNR : 15.06 dB
 Recovery SNR : 32.77 dB
 Digital Mod Used : QPSK (SNR 8-15 dB)
 Decoding Method : Parity Decode
 Final BER : 0.00000
 Errors Corrected : 0
 Retransmissions : 0

OUTPUT



REAL-WORLD APPLICATIONS & LIMITATIONS

4G/5G Systems

Adaptive communication systems are essential for **4G and 5G networks**, significantly improving data throughput and reliability, allowing for seamless connectivity and enhanced user experiences in mobile communications.

WiFi Communications

In WiFi networks, **adaptive techniques optimize bandwidth usage**, ensuring that multiple devices can connect simultaneously without sacrificing performance, which is crucial for maintaining high-speed internet access.

IoT Applications

Adaptive communication systems enhance **Internet of Things (IoT) applications**, enabling devices to communicate effectively in diverse environments while adjusting to varying signal conditions for improved data transmission reliability.

Signal Processing

These systems play a pivotal role in **signal processing**, allowing devices to accurately interpret and process incoming signals, which is vital for applications in telecommunications and data analysis.

Satellite Communication

By demonstrating modern communication techniques, adaptive systems pave the way for **innovations in technology**, promoting advancements that cater to the growing demand for faster and more reliable connectivity.

Additional Notes

Understanding these applications helps illustrate the importance of **adaptive communication systems** in today's technology-driven world, showcasing their vital role in enhancing communication efficiency and performance.

Advantages of This System

Handles all 6 signal types in one unified pipeline

Full 12-step end-to-end communication demonstrated

No advanced MATLAB toolboxes required

Easy to understand, extend, and demonstrate

BER feedback loop enables self-optimization

Limitations

MATLAB-based simulation only — not real RF hardware

Simplified AWGN channel model (no fading / multipath)

Fixed thresholds used for adaptive switching

ESP32 data rate limited by serial communication speed

CONCLUSION & FUTURE SCOPE

Project Summary

Developed an Adaptive Multi-Source End-to-End Communication System that processes signals from Audio, Mathematical equations, and ESP32 sensors — performing a complete 12-step communication pipeline.

Key Achievement : The system automatically detects signal type and adapts its processing for efficient recovery — including adaptive modulation, error correction, and BER-based feedback.

Impact of the System

Improves signal quality and reliability

Reduces BER through adaptive feedback

Adapts to changing channel environments

Demonstrates modern communication techniques used in real-world systems

Technical Strength

Full pipeline: Signal Generation → Processing → Transmission → Recovery. Works without advanced toolboxes. Easy to understand and demonstrate.

FUTURE SCOPE

Machine Learning-based automatic signal detection

Real hardware RF implementation (SDR / FPGA)

Advanced channel models: fading, multipath

Real-time full communication system deployment

References

Haykin, S. (2001). Communication Systems. Wiley.

Proakis, J.G. (2007). Digital Communications. McGraw-Hill.

Sklar, B. (2001). Digital Communications. Prentice Hall.

Rappaport, T.
S. (2002). Wireless Communications. Prentice Hall.

MATLAB Documentation — Signal Processing Toolbox.

ESP32 Technical Reference Manual — Espressif Systems.



THANK YOU